

Konstantin Lucius Regenhardt

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Bachelor of Science Umweltinformatik*
am Department Fachbereich 2
der Fakultät Technik und Informatik
der Hochschule für Technik und Wirtschaft Berlin

Betreuender Prüfer: Prof. Dr. Jochen Wittmann
Zweitgutachter: Prof. Dr. Zaphod Beeblebrox

Eingereicht am: 01.08.2026

Konstantin Lucius Regenhardt

Thema der Arbeit

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Stichworte

Kurzzusammenfassung

In der Softwareentwicklung bieten proprietäre Low-Code-Plattformen wie Katalon einen schnellen Einstieg in die Testautomatisierung. Mit steigender Projektkomplexität führen diese jedoch häufig zu einem massiven Kostenanstieg durch unflexible Lizenzmodelle für fortgeschrittene Funktionen. Da die Testlogik und Datenobjekte in proprietären Formaten gespeichert sind, stehen Nutzer vor dem Dilemma, entweder hohe Gebühren zu zahlen oder den Verlust der bisherigen Entwicklungsarbeit bei einem Plattformwechsel hinzunehmen. Ziel dieser Bachelorarbeit ist die Entwicklung eines programmatischen Migrationspfades von Katalon zu einer Open-Source-Alternative, geschrieben in Selenium. Dabei soll die manuelle Neuerstellung der Test-Suites durch eine automatisierte Transformations-Pipeline ersetzt werden, um die bisherige Arbeit nicht zu verlieren. Es wird ein Migrations-Algorithmus entworfen, der auf Regular Expressions und Python-Methoden basiert. Dieser scannt die proprietären Strukturen, übersetzt die bestehende Testlogik von Groovy nach Python und bildet die internen Datenobjekte sowie Referenzdateien auf eine neue, Open-Source Projektstruktur im Selenium-Framework ab. Das Ergebnis ist ein funktionsfähiger Prototyp eines Migrations-Tools. Dieses Tool ermöglicht den Transfer von Test-Suites in ein erweiterbares Open-Source-Projekt. Dabei bleibt die Integrität der Tests gewahrt und die Abhängigkeit von Lizenzgebühren wird vollständig eliminiert. Die Arbeit zeigt auf, wie durch automatisierte Code-Migration und -Transformation der Wechsel von proprietären „Low-Code“ zu Open-Source „Pro-Code“ gelingen kann. Die Ergebnisse bieten Nutzern eine gute Möglichkeit für den Ausstieg aus proprietären Test-Ökosystemen.

Konstantin Lucius Regenhardt

Title of Thesis

From Proprietary Test Automation to Open Execution: Design and Evaluation of a Katalon-to-Selenium/Pytest Migration Tool

Keywords

Abstract

In software development, proprietary low-code platforms like *Katalon* offer a fast initial entry point into test automation. However, as project complexity scales, these systems frequently lead to a massive escalation of costs due to inflexible licensing frameworks. This thesis addresses this „Vendor Lock-In“ problem by establishing an automated transformation pipeline that migrates test suites from closed Katalon structures into an open-source, flexible framework written in Python using *Selenium*.

A core migration algorithm based on Python execution logic and Regular Expressions (Regex) handles the automated parsing of proprietary formats, translations of test scripts from *Groovy* to *Python*, and mapping of object repositories. The evaluation showcases a fully functional prototype that successfully eliminates license fee dependencies while completely preserving test suite runtime integrity.

Abkürzungs- und Begriffsverzeichnis

API – Application Programming Interface: Eine definierte Schnittstelle, über die Softwarekomponenten miteinander kommunizieren. Selenium stellt eine API bereit, über die Tests den Browser steuern.

CI – Continuous Integration: Ein Softwareentwicklungsprinzip, bei dem Code-Änderungen regelmäßig in ein gemeinsames Repository integriert und automatisch gebaut und getestet werden. Automatisierte Testsuiten sind ein zentrales Werkzeug im CI-Prozess.

Pytest Fixture: Ein Mechanismus in Pytest, der wiederverwendbare Test-Vorbedingungen und -Nachbedingungen definiert. Fixtures werden in der Datei `conftest.py` deklariert und stehen allen Tests im Projekt automatisch zur Verfügung, ohne explizit importiert werden zu müssen. Typische Anwendungen sind das Erstellen eines WebDriver-Objekts oder das Einrichten von Testdaten. 7

Groovy: Eine dynamisch typisierte Skriptsprache, die auf der JVM läuft und vollständig mit Java interoperabel ist. Katalon Studio verwendet Groovy als Skriptsprache für Testskripte. In dieser Arbeit wird Groovy-Code mittels Regex-Transformation in Python übersetzt. 4, 6, 9

IDE – Integrated Development Environment: Eine Software-Anwendung, die Entwicklern eine umfassende Umgebung für die Softwareentwicklung bietet. Typischerweise umfasst sie einen Code-Editor, Debugger, Build-Werkzeuge und oft eine grafische Projektansicht. Katalon Studio ist eine solche IDE, die speziell auf Testautomatisierung ausgelegt ist. 5, 5, 5, 7

JVM – Java Virtual Machine: Eine Laufzeitumgebung, die plattformunabhängige Ausführung von Java-Bytecode ermöglicht. Groovy, die Skriptsprache von Katalon Studio, läuft auf der JVM. 4

Locator: Ein Ausdruck, mit dem Selenium ein HTML-Element auf einer Webseite identifiziert. Gängige Locator-Strategien sind ID, CSS-Selektor, XPath und Name. Katalon speichert Locatoren im Object Repository; bei der Migration werden sie in Selenium-kompatible By-Strategien überführt. 8

Object Repository: Eine zentrale Datenstruktur in Katalon Studio, die alle Test Objects (z. B. Schaltflächen, Eingabefelder) mit ihren Lokalisierungsstrategien speichert. Jedes Objekt enthält Eigenschaften wie XPath, CSS-Selektor oder ID, anhand derer Selenium das Element im DOM der Webanwendung identifiziert. 4, 8

PoC – Proof of Concept: Ein Prototyp oder eine Machbarkeitsstudie, die zeigt, dass ein Konzept oder eine Idee praktisch umsetzbar ist. In dieser Arbeit bezeichnet PoC die erste Umsetzung eines Migrationsskripts zur Überprüfung der Machbarkeit.

Regex – Regular Expression: Ein formaler Ausdruck zur Beschreibung von Zeichenketten-Mustern. In dieser Arbeit werden Regular Expressions zur Transformation von Groovy-Syntax in Python-Syntax eingesetzt. 9

SUT – System Under Test: Das zu testende System — in dieser Arbeit die Webanwendung, auf die die automatisierten Selenium-Tests angewendet werden.

Test Object: Ein in Katalon Studio definiertes Objekt, das ein HTML-Element auf einer Webseite repräsentiert. Test Objects enthalten Informationen wie Name,

Beschreibung und Locator. Sie werden im Object Repository gespeichert und in Testskripten referenziert. 7

Transpilation: Der Prozess der automatischen Übersetzung von Quellcode einer Programmiersprache in eine andere. Im Unterschied zur Kompilation bleibt das Abstraktionsniveau erhalten. In dieser Arbeit bezeichnet Transpilation die Übersetzung von Groovy-Testskripten nach Python.

UI – User Interface: Die Benutzerschnittstelle einer Anwendung, über die Nutzer mit dem System interagieren. Im Kontext dieser Arbeit bezeichnet UI die grafische Oberfläche, die durch automatisierte Tests validiert wird. 4, 5

Vendor Lock-in: Die technische oder wirtschaftliche Abhängigkeit eines Unternehmens von einem einzelnen Anbieter, die einen Wechsel zu Alternativen erschwert oder verteuert. Im Kontext dieser Arbeit bezeichnet Vendor Lock-in die Bindung an Katalon Studio durch proprietäre Dateiformate und lizenzpflichtige Funktionen. 3

WebDriver: Eine standardisierte API (W3C-Standard), über die Programme einen Webbrowser programmatisch steuern können. Selenium WebDriver ist die Referenzimplementierung und bildet die Grundlage aller in dieser Arbeit generierten Tests. 2

Inhaltsverzeichnis

Abkürzungs- und Begriffsverzeichnis	IV
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Ziel & Aufbau der Arbeit	2
2 Grundlagen	2
2.1 Automatisiertes Testen	2
2.2 Vendor Lock-in bei proprietären Plattformen	3
2.3 Low-Code-Plattformen im Testbereich	3
3 Strukturanalyse	4
3.1 Katalon Projektstruktur	4
3.1.1 Elementare Ordner und Dateien	4
3.1.2 Verschachtelungen in der Katalon Struktur	5
3.2 Selenium/Pytest Projektstruktur	6
3.2.1 Testskripte & Variablen	6
3.2.2 Testdaten & Einbindung	6
3.2.3 Pytest Fixtures & Konfiguration	7
4 Konzeption der Migrationspipeline	7
4.1 Architekturüberblick	7
4.2 Transformationsablauf	8
4.2.1 Syntaktische Transformation: Groovy zu Python	8
4.2.2 Semantische Transformation: Katalon Objects zu Selenium Locators	8
4.2.3 Strukturelle Transformation: Projektlayout	9
4.3 Regex-basierte Transformation	9
4.4 Datenfluss durch die Pipeline	9
4.5 Detailliertes Aktivitätsdiagramm der Test-Transformation	9

5	Implementierung des Prototyps	10
5.1	Refaktorierte Projektstruktur	10
5.2	Pipeline-Phasen (Datenfluss)	12
5.3	Kernmodule und ihre Funktionen	13
5.3.1	test_transpiler.py — Groovy→Python Transpildfähigkeit	13
5.3.2	test_assembler.py — Pytest Code-Generierung	13
5.3.3	object_repo_converter.py — Object Repository Migration	13
5.3.4	global_vars_generator.py — Globale Variablen	14
5.3.5	runtime/base_test.py — Selenium Base Class	15
5.3.6	runtime/katalon_helpers.py — Katalon-kompatible Helfer	16
5.4	Fehlerbehandlung und Robustheit	16
5.5	Konfigurierbarkeit und Erweiterung	16
5.5.1	base_test.py — Pytest-Basis-Klasse	17
6	Evaluation & Results	17
7	Discussion & Future Work	17
8	Conclusion / Zusammenfassung	17
9	Appendix / Anhang	18
10	Declaration of Originality / Eigenständigkeitserklärung	18
	Quellenverzeichnis	19
	Erklärung zur selbstständigen Bearbeitung	20

Abbildungsverzeichnis

Abbildung 1 Vereinfachter Datenfluss durch die Migrationspipeline. Jede Phase produziert Intermediate Representations, die in die nächste Phase fließen.	9
---	---

Tabellenverzeichnis

Tabelle 1 Runtime performance and execution velocity comparisons between the legacy system and the migrated prototype. (Source: Compiled from test logs) 17

1 Einleitung

1.1 Motivation

Automatisiertes Testen ist eine zentrale Komponente moderner Software-Qualitätssicherung. In vielen Teams werden Testautomatisierungs-Frameworks eingesetzt, um Regressionstests zu beschleunigen und das Vertrauen in Veröffentlichungen zu stärken[1]. Anfangs nutzte unser Team Katalon Studio als IDE um neben der Entwicklung einer internen Geschäftssoftware auch eine grundlegende Testinfrastruktur aufzubauen. Im Laufe der Zeit wuchs die Infrastruktur stark an und umspannte den Großteil der Software.

Da sich die Anforderungen des Projekts weiterentwickelten, benötigte das Team fortgeschrittenere Automatisierungs-Techniken, wie automatisiertes Berechnen von Datums-Einträgen und eine tiefere Kontrolle über die Testausführung und das Debugging. In dieser Phase traten erste Blockaden auf: Erweiterte Funktionen, die im Programmieren als elementar gesehen werden, erforderten zusätzliche Lizenzen, was erhebliche Kosten verursachte und die Skalierung der Infrastruktur einschränkte. Dies kristallisierte sich schnell als ein blockierendes Problem heraus da die Preise in keinem Verhältnis zu den Leistungen standen und über längere Zeit nicht tragbar waren. Als Lösungsansatz wurde eine Migration zu einem Open-Source Projekt vorgeschlagen.

Angesichts der erheblichen bestehenden Investitionen in eine Katalon-basierte Test-Umgebung war ein vollständiges Neuschreiben der aller gesammelten Tests von Grund auf, nicht machbar. Deswegen wurde ein Migrationsansatz als PoC untersucht, um die zuvor erstellten Tests und alles Zugehörige zu erhalten und gleichzeitig den Übergang zu einer offenen und erweiterbaren Open-Source Struktur zu ermöglichen.

1.2 Problemstellung

Das Kernproblem das in dieser Arbeit behandelt wird, ist die Frage wie ein bestehendes Katalon-Projekt beim Wechsel in eine lizenzunabhängige Open-Source Umgebung beibehalten und wiederverwendet werden kann. Eine manuelle Migration von Katalon-Tests zu Selenium/Pytest ist extrem zeitaufwendig und bei einer großen Testmenge praktisch unerreichbar. Das liegt daran, dass neben der reinen Skriptübersetzung eine Menge an zusätzlichen Abhängigkeiten und Konfigurationen berücksichtigt werden müssen. Ein Katalon-Projekt besteht nicht nur aus Testskripten, sondern auch aus einer Vielzahl von Assets wie Objekt-Repositories, Variablen-Definitionen, globalen Profilen und Datenabhängigkeiten. Diese sind jeweils in proprietären Formaten gespeichert und müssten manuell „ausgegraben“ und zusammengeführt werden. Eine manuelle Migration würde daher nicht nur die Übersetzung von Groovy nach Python erfordern, sondern auch die Rekonstruktion der gesamten Projektstruktur und die Anpassung an die Open-Source-Frameworks.

Die technische Herausforderung liegt nicht nur in der syntaktischen Konvertierung des Testcodes, sondern in der End-to-End-Transformation eines Testautomatisierungsprojekts in ein ausführbares und skalierbares alternatives Format.

1.3 Ziel & Aufbau der Arbeit

Das Ziel dieser Arbeit ist es, einen automatisierten Migrator zu entwerfen, zu implementieren und zu evaluieren, der Katalon-basierte Testprojekte in eine Python-Selenium/Pytest-Projektstruktur konvertiert.

Um dieses Ziel zu erreichen wird in dieser Arbeit zuerst werden die Struktur eines Katalon-Projektes und die eines Python-Selenium/Pytest-Projektes analysiert. Dann wird Stück für Stück ein Migrationsalgorithmus entworfen, der die proprietären Formate des einen in offene nutzbare Formate des anderen überführt. Anschließend wird ein Prototyp implementiert, der die Transformation automatisiert und die Integrität der Tests sicherstellt. Abschließend werden die Ergebnisse evaluiert und diskutiert, um die Effektivität des Migrationsprozesses zu bewerten.

2 Grundlagen

2.1 Automatisiertes Testen

Softwaretests sind ein grundlegender Bestandteil der Qualitätssicherung in der Softwareentwicklung. Ihr Ziel ist es, sicherzustellen, dass ein System korrekt funktioniert und definierte Anforderungen erfüllt. Dabei unterscheidet man grob zwischen manuellen und automatisierten Tests.

Bei manuellen Tests führt eine Person die Testschritte selbst aus und bewertet das Ergebnis. Dieser Ansatz ist bei kleinen, seltenen oder testhaften Überprüfungen praktikabel, skaliert jedoch schlecht: Mit wachsender Softwarekomplexität steigt der Zeitaufwand für manuelle Regressionstests exponentiell.

Automatisiertes Testen bezeichnet das maschinengestützte Ausführen von Testfällen anhand von Testskripten. Die Skripte definieren Eingaben, Aktionen und erwartete Ergebnisse. Wesentliche Vorteile automatisierter Tests sind[2]:

- **Wiederholbarkeit:** Dieselben Tests können beliebig oft und konsistent ausgeführt werden.
- **Geschwindigkeit:** Große Test-Suiten lassen sich in kurzer Zeit ausführen.
- **Regressionssicherheit:** Nach Codeänderungen kann schnell geprüft werden, ob bestehende Funktionalität noch korrekt arbeitet.
- **CI/CD-Integration:** Automatisierte Tests können direkt in Deployment-Pipelines eingebunden werden.

Ein häufig eingesetztes Verfahren ist der **End-to-End-Test** (E2E-Test), bei dem eine Anwendung aus Nutzerperspektive durch die Benutzeroberfläche getestet wird. Werkzeuge wie die Selenium WebDriver ermöglichen solche Tests für webbasierte Anwendungen, indem sie einen Browser programmatisch steuern.

2.2 Vendor Lock-in bei proprietären Plattformen

Vendor Lock-in beschreibt die Abhängigkeit eines Nutzers oder einer Organisation von einem bestimmten Anbieter, sodass ein Wechsel zu einer Alternative mit erheblichem Aufwand oder hohen Kosten verbunden ist[3]. Diese Abhängigkeit entsteht häufig durch proprietäre Dateiformate, plattformspezifische Programmiersprachen oder APIs, die außerhalb des jeweiligen Ökosystems nicht verwendet werden können.

Im Bereich der Testautomatisierung zeigt sich Vendor Lock-in beispielsweise dann, wenn:

- Testskripte in einer plattformeigenen Skriptsprache verfasst sind und nicht ohne Weiteres auf andere Frameworks übertragen werden können,
- interne Datenstrukturen wie Objekt-Repositorys oder Profil-Konfigurationen in proprietären Formaten gespeichert sind,
- bestimmte Funktionen nur in kostenpflichtigen Lizenzstufen verfügbar sind.

Die Konsequenz ist, dass Teams entweder steigende Lizenzkosten akzeptieren oder bei einem Plattformwechsel einen erheblichen Teil ihrer bisherigen Arbeit verlieren. Für das in dieser Arbeit beschriebene Projekt manifestierte sich dieses Problem konkret: Grundlegende Entwicklungsfunktionen wie das Debugging über die CLI erforderten teurere Lizenzpakete, was die langfristige Nutzung der Plattform unwirtschaftlich machte.

2.3 Low-Code-Plattformen im Testbereich

Low-Code-Plattformen sind Entwicklungsumgebungen, die komplexe technische Operationen durch grafische Oberflächen und vorgefertigte Bausteine abstrahieren. Im Bereich der Testautomatisierung bieten sie einen niedrighschwelligen Einstieg: Tests können über eine GUI aufgezeichnet oder aus einem Katalog von vordefinierten Aktionen zusammengestellt werden, ohne tiefgehende Programmierkenntnisse vorauszusetzen.

Diese Eigenschaft macht Low-Code-Tools attraktiv für Teams in der frühen Phase der Testautomatisierung. Mit wachsender Anforderungskomplexität stoßen sie jedoch an Grenzen[4]:

- Individuelle Logik, die über vorgefertigte Bausteine hinausgeht, ist schwer oder gar nicht umsetzbar.
- Die Testlogik ist eng an die Plattform gebunden und schlecht in externe Versionskontrollsysteme integrierbar.
- Skalierbarkeit und Anpassbarkeit sind durch das Plattformmodell begrenzt.

Im Gegensatz dazu stehen sogenannte **Pro-Code**-Ansätze, bei denen Tests vollständig in einer allgemeinen Programmiersprache wie Python geschrieben werden. Frameworks wie Selenium und Pytest bieten dabei maximale Flexibilität, erfordern aber entsprechende Programmierkenntnisse. Der Übergang von Low-Code zu Pro-Code ist inhaltlich des Kerns dieser Arbeit.

3 Strukturanalyse

3.1 Katalon Projektstruktur

Ein Katalon Studio Projekt besteht, aus dem Blickwinkel des IDE-Benutzers, unter Anderem aus Tests, einem „Object Repository“, globalen Variablen in Profilen, testeigenen Variablen, eingebundenen Test-Daten. Diese werden in einer hierarchischen Ordnerstruktur gespeichert, die von der IDE verwaltet wird.

3.1.1 Elementare Ordner und Dateien

3.1.1.1 Test Cases & Variablen

Die Testskripte in Katalon sind in Groovy geschrieben, einer dynamischen Sprache, die auf der Java Virtual Machine (JVM) läuft. Sie werden mit einer Vielzahl von eingebauten Keywords und Funktionen geliefert, die speziell für Testautomatisierung entwickelt wurden. Variablen können sowohl global als auch lokal definiert werden, wobei globale Variablen in Profilen gespeichert sind und in allen Tests zugänglich sind.

3.1.1.2 Object Repository

Das Object Repository ist ein zentrales Element in der Katalon Umgebung, das das Speichern von „User Interface (UI)“-Elementen ermöglicht. Es speichert die Eigenschaften von HTML-Elementen, die während der Testausführung verwendet werden. Dafür kann der Benutzer die Elemente mit einem Tool das Katalon Studio bereitstellt auf der Webseite auswählen und im Object Repository generieren lassen. Diese Elemente können in verschiedenen Tests wiederverwendet werden, was die Wartung und Skalierbarkeit der Tests erleichtert.

3.1.1.3 Globale Variablen & Profile

Jede/r User/in kann in Katalon eigene globale Variablen definieren, die in verschiedenen Test Cases verwendet werden können. Diese Variablen werden in Profilen gespeichert, welche unterschiedliche Benutzende oder Umgebungen repräsentieren können. Profile ermöglichen es, Tests in verschiedenen Konfigurationen auszuführen, ohne den Testcode selbst ändern zu müssen. Hat ein Profil die Variable „URL“ mit dem Wert „https://staging.example.com“ und ein anderes Profil die gleiche Variable mit dem Wert „https://production.example.com“, kann der Test in beiden Umgebungen ausgeführt werden, indem einfach das entsprechende Profil ausgewählt wird.

3.1.1.4 Testdaten & Einbindung

Katalon kann Testdaten aus verschiedenen Quellen, wie Excel-Dateien, CSV-Dateien oder Datenbanken einbinden. Diese Testdaten können in den Testskripten referenziert werden, um die Tests mit unterschiedlichen Eingabewerten auszuführen.

3.1.1.5 Custom Keywords

Benutzer/Innen können eigene Methoden, sogenannte „Custom Keywords“ erstellen, die in mehreren Test Cases wiederverwendet werden können. Diese Keywords werden in Groovy geschrieben und ermöglichen es, komplexere Logik für Tests zu nutzen und diese wiederverwendbar zu machen.

3.1.2 Verschachtelungen in der Katalon Struktur

Alle Dateien und Ordner die man in der Katalon Integrated Development Environment (IDE) erstellt, werden durch den Viewport übersichtlich dargestellt. Sobald man sich die Ordnerstruktur eines Katalon-Projekts jedoch außerhalb der IDE anschaut, erkennt man, dass die Testskripte nicht nur eine einzelne Datei sind, wie im Programmieren üblich.

3.1.2.1 Verschachtelung der Test Cases

Ein „Test Case“ besteht in Katalon aus mehreren Dateien, die zusammenarbeiten. Folgt man der Katalon-Ordnerstruktur im Explorer unter „Test Cases“, so wie sie in der IDE zu sehen ist landet man bei einer .tc-Datei. Öffnet man diese, erkennt man schnell, dass es sich dabei um eine umbenannte .xml-Datei handelt. Sie enthält Meta-Daten, wie eine Beschreibung, den Testnamen, Tags, Kommentare, eine GUID und die eigentlichen Werte der Test-eigenen Variablen. Es gibt keine offensichtliche Referenz auf die eigentliche Testlogik. Diese ist in einer weiteren Datei gespeichert. Um sie zu finden muss man den Ordner „Scripts“ öffnen, dann den Pfad des „Test Cases“ spiegeln. Dort befindet sich ein Ordner der den Testnamen trägt und dieser enthält dann eine .groovy-Datei, die den Namen „Script“ kombiniert mit einer zufälligen Nummer trägt. Die Nummer steht in keiner Verbindung zu der .tc-Datei. In dieser .groovy-Datei befindet sich dann die eigentliche Testlogik. Greift der Test auf eine Variable zu, wird im Skript der „rohe“ Name der Variable ausgeschrieben. „Roh“ bedeutet, dass der Name im Code keinen Datentyp hat und auf nichts im Test verweist. Die Variable wird im Test weder deklariert, noch initialisiert.

3.1.2.2 Verschachtelung der Globalen Variablen

Wenn der Test eine Globale Variable verwendet, zeigt sich dies im Code als „GlobalVariable.VARIABLENNAME“. Der Wert der Variable ist nicht im Skript zu finden sondern in der Profil-Datei mit der Endung .glbl, die im Ordner „Profiles“ gespeichert ist. Diese Datei ist ebenfalls eine .xml-Datei und enthält die Variablen als Eintrag in einer „GlobalVariableEntity“ - ein Eintrag der die Variable beschreibt. Diese Einträge enthalten den tatsächlichen Wert und Meta-Daten, bestehend aus einer Beschreibung, dem Variablennamen, dem Datentyp und einem Boolean der beschreibt ob der Wert „protected“ ist. Ist er „protected“, wird der Wert als sensibel behandelt und in der UI und bei der Bearbeitung nur mit dem Stern-Charakter maskiert angezeigt.

3.1.2.3 Verschachtelung der Testdaten

Sobald man mit Katalon Studio ein Dateiformat mit Daten einbinden möchte, erstellt Katalon Studio eine .dat-Datei im Ordner „Data Files“ die den gleichen Namen wie die eingebundene Datei trägt. Die Datei enthält ebenfalls XML-Strukturen, nur ein paar Meta-Daten enthalten. Unter anderem den Pfad zur eingebundenen Datei, die Dateiarart, ggf. die Trennzeichen und ob der Pfad Projektintern oder -extern ist. Die eigentlichen Daten sind nicht in der .dat-Datei enthalten, sondern in der eingebundenen Datei.

3.1.2.4 Verschachtelung der Custom Keywords

Der Großteil der „Custom Keywords“-Logik wird im Ordner „Keywords“ und selbst erstellten Unterordnern gespeichert, und ist in Groovy in der Form „KlassenName.groovy“ geschrieben. Diese Skripte enthalten die eigentlichen Klassen und Methoden die in den Tests aufgerufen werden. Im Ordner „Libs“ gibt es die „CustomKeywords.groovy“-Datei. Diese definiert statische Weiterleitungen mit dem gleichen Namen wie die Klassen im Ordner „Keywords“. Diese Methoden rufen die eigentlichen Methoden in den Klassen auf und machen sie dadurch in jedem Test aufrufbar.

3.2 Selenium/Pytest Projektstruktur

Ein Projekt, das Selenium und Pytest für die Testautomatisierung nutzt, ist im Gegensatz zu Katalon nicht an eine proprietäre Projektstruktur gebunden. Pytest erwartet nur eine nachvollziehbare Dateioorganisation durch Konventionen, gültige Python-Module und eine Konfiguration im Projektwurzverzeichnis, der „root“, in Form einer pyproject.toml oder pytest.ini[5]. In einem reinen Testprojekt enthält das Repository dabei keinen Anwendungscode, sondern ausschließlich Testfälle, gemeinsam genutzte Hilfsmodule, Konfigurationsdateien und Testdaten. Typischerweise existiert ein zentrales „tests“-Verzeichnis, das bei wachsender Projektgröße weiter in Teilbereiche der zu testenden Anwendung gegliedert wird, beispielsweise in UI-Tests und API-Tests[6]. Das aus Python-Paketprojekten bekannte src-Layout ist in diesem Fall nicht zwingend erforderlich. Laut Python Packaging User Guide dient das src-Layout vor allem dazu, importierbaren Anwendungscode klar von der Projekt-root zu trennen[7].

3.2.1 Testskripte & Variablen

Die eigentlichen Testfälle folgen den von Pytest vorgesehenen Namenskonventionen wie test_NAME.py oder NAME_test.py[6]. Anders als bei Katalon besteht ein Testfall dabei in der Regel aus genau einer Python-Datei, in der die Testlogik direkt lesbar ist. Wiederverwendbare Browser-Interaktionen, Element-Selektoren oder Methoden können in separate Hilfsmodule ausgelagert werden. Diese heißen oft „Helper“- oder „Utility“-Funktionen.

Variablen werden in einem solchen Projekt nicht über proprietäre Profil- oder Metadateien verwaltet, sondern über normale Python-Programmierstrukturen. Typisch sind Parametrisierungen in den Testfunktionen, Rückgabewerte von Fixtures, Umgebungsvariablen oder externe Konfigurationsdateien. Dadurch bleibt nachvollziehbar, woher ein Wert stammt und an welcher Stelle er in den Test eingebunden wird.

3.2.2 Testdaten & Einbindung

Testdaten werden häufig in einem eigenen Verzeichnis wie z. B. „data“, „resources“ oder „testdata“ abgelegt und dann bei Bedarf direkt von den Tests oder von Hilfsmodulen eingelesen. Dabei kann es sich beispielsweise um JSON-, CSV-, XML- oder Excel-Dateien handeln, jedoch ist praktisch jedes Format nutzbar. Im Unterschied zu Katalon entsteht hierbei keine zusätzliche proprietäre Metadatei, die nur einen Verweis auf die eigentliche Datenquelle enthält.

Stattdessen sind Dateipfade, Ladeverhalten und Weiterverarbeitung unmittelbar im Python-Code oder in den zugehörigen Hilfsfunktionen sichtbar.

3.2.3 Pytest Fixtures & Konfiguration

Eine zentrale Rolle in Pytest nimmt die Datei `conftest.py` ein. Die dort definierten Pytest Fixture sind wiederverwendbare Testkomponenten und stehen allen Tests automatisch zur Verfügung und müssen nicht explizit importiert werden[8]. Auf diese Weise lassen sich gemeinsam genutzte Setups, etwa die Erstellung eines WebDriver-Objekts, Logins, temporäre Verzeichnisse oder standardisierte Testdaten, an einer gemeinsamen Stelle sammeln.

Die allgemeine Projektkonfiguration liegt typischerweise im `root`-Verzeichnis des Repositories. Dort können unter anderem Testpfade, Standardoptionen für den Testlauf oder Marker definiert werden[5].

Für ein reines Selenium/Pytest-Testprojekt ergibt sich damit eine vergleichsweise schlanke Struktur aus Testskripten, gemeinsam genutzten Fixtures, optionalen Hilfsmodulen und separaten Testdaten. Im Vergleich zur Katalon-Struktur sind sowohl die Pfade als auch die Abhängigkeiten zwischen Testlogik, Konfiguration und Daten deutlich direkter ablesbar.

4 Konzeption der Migrationspipeline

Eine ursprüngliche Idee, als die Aufgabe der Migration in ein anderes Ökosystem aufkam, war es ein Selenium/Pytest-Projekt zu schreiben, das Katalon-Tests direkt lesen und ausführen kann. Diese Idee wurde nach Rücksprache mit anderen Entwicklern jedoch schnell verworfen, da die proprietären Formate und die Katalon-spezifische Logik zu komplex waren, um sie direkt in einem Open-Source-Framework auszuführen. Stattdessen wurde ein Ansatz gewählt, der die Katalon-Testlogik in eine neue, offene Struktur transformiert. Durch diese komplette Trennung von Katalon-Strukturen konnten Wartbarkeit und Erweiterbarkeit gesichert werden. Für den nächsten Ansatz wurde erst eine kleine Pipeline gebaut, die versuchte, häufig vorkommende Methoden zu übersetzen. Dieser Ansatz zeigte sehr schnell, dass die Katalon-Struktur im Hintergrund der IDE viel komplexer und verschachtelter ist, als es auf den ersten Blick scheint. Mit der Zeit entwickelten sich erst Regex-basierte Methoden zum Erkennen aller Katalon Funktions-Aufrufe, dann Weitere zum Erkennen von Pfaden zu Test Objects, dann zum Erkennen von Variablen und schließlich zum Erkennen von Globalen Variablen und deren Werten. Mit der Zeit entstand so eine großläufige Migrationspipeline, die die zentralen Aspekte einer Katalon-Struktur erkennt, diese transformiert und in einer neuen Selenium/Pytest-Struktur abspeichert.

4.1 Architekturüberblick

Die Migrationspipeline enthält viele kleine Module die zusammen Schritt für Schritt alle Inhalte transformieren. Zusammen ergeben sich fünf Kernfunktionen, die zusammen den kompletten Transformationsprozess abbilden. Der „Scanner & Parsing“-Schritt fungiert als Einstiegspunkt. Dabei scannt und extrahiert man alle Projektkomponenten wie Test Cases, Object Repository,

Variablen und Testdaten systematisch aus dem alten Projekt. Dann der „Transpilations“-Schritt, mit der Aufgabe Regex-basierte Leseregeln auf den Groovy-Syntax anzuwenden und den ursprünglichen Code schrittweise zu erkennen und zu übersetzen. Eng verzahnt damit, passiert währenddessen das Mapping und die Validation, um Katalon-Datentypen auf ihre Python-Äquivalente abzubilden und gleichzeitig die Integrität aller Transformationen zu validieren. Danach folgt die Code-Generierung. Hier wird aus den transformierten Strukturen, gültiger Selenium-Pytest-Code der unmittelbar ausführbar ist generiert. Als letztes kommt der kombinierende „Project Assembly“-Schritt, der alle generierten Dateien in eine korrekte und wartbare Selenium/Pytest-Projektstruktur schreibt. Diese Pipeline-Architektur gewährleistet, dass alle wichtigen Element eines Katalon-Projektes systematisch, nachvollziehbar und wartbar transformiert werden.

4.2 Transformationsablauf

4.2.1 Syntaktische Transformation: Groovy zu Python

Die Groovy-Syntax wird durch Regex-Pattern-Matching in Python-Syntax überführt. Aus der Implementierung ergeben sich folgende Transformationen:

Beispiel 1: WebUI-Methodenaufrufe

Katalon (Groovy):

```
WebUI.callTestCase(findTestCase('Util/login'), [:], FailureHandling.STOP_ON_FAILURE)
WebUI.verifyTextPresent('View All Users', false)
WebUI.click(findTestObject('All_Users/view_all_users_btn'))
WebUI.setText(findTestObject('All_Users/search_input'), david)
WebUI.verifyTextPresent(GlobalVariable.user4name, false)
```

Generierter Python-Code:

```
# findTestCase('Util/login')[:]FailureHandling.STOP_ON_FAILURE
assert 'View All Users' in self.driver.page_source
kh.find_katalon_test_object(self.driver, 'All_Users/view_all_users_btn').click()
kh.find_katalon_test_object(self.driver, 'All_Users/search_input').clear()
kh.find_katalon_test_object(self.driver, 'All_Users/
search_input').send_keys(vars.david)
assert GlobalVariable.USER4NAME in self.driver.page_source
```

4.2.2 Semantische Transformation: Katalon Objects zu Selenium Locators

Das Katalon Object Repository speichert Testobjekte als XML-Strukturen mit verschachtelten Eigenschaften. Diese werden in „Locator“-Strategien für Selenium überführt:

- Katalon Properties (id, name, xpath, css) → Selenium By-Strategien (By.ID, By.XPATH, By.CSS_SELECTOR)
- Mehrschichtige Objekthierarchien → Flache Locator-Listen mit Parent-Child-Referenzen
- Dynamische Property-Substitution → Runtime Variable Substitution in Pytest Fixtures

4.2.3 Strukturelle Transformation: Projektlayout

Die hierarchische Katalon-Struktur wird in eine flache, standardisierte Python-Struktur transformiert:

Katalon Structure:	→	Selenium/Pytest Structure:
├─ Tests/	→	tests/
├─ Object Repository/	→	tests/locators/
├─ Profiles/	→	tests/config/
└─ Data Files/	→	tests/data/

4.3 Regex-basierte Transformation

Das Werkzeug nutzt „Regular Expression (Regex)“ zur Transformation von „Groovy“-Syntax. Ein Beispiel-Pattern für Katalon WebUI-Befehle:

```
Pattern: WebUI\.(\\w+)\(((\\^,\\+),\\s*(\\.+?)\\)
Replacement: selenium_action("$1", locator="$2", args=$3)
```

Dies transformiert `WebUI.click(testObject, [timeout: 3])` zu einer äquivalenten Python-Selenium-Aufrufen, die über eine Wrapper-Funktion abstrahiert werden.

4.4 Datenfluss durch die Pipeline

```
[Katalon Project] → [Parser: Extract] → [Transformer: Groovy→Python] → [Mapper: Objects→Locators] → [Generator: Write Code] → [Output: Pytest Structure]
```

Abbildung 1: Vereinfachter Datenfluss durch die Migrationspipeline. Jede Phase produziert Intermediate Representations, die in die nächste Phase fließen.

4.5 Detailliertes Aktivitätsdiagramm der Test-Transformation

Das folgende Diagramm zeigt den genauen Transformationsablauf eines Katalon-Tests zu einem Python-Pytest-Test. Es illustriert die zentralen Phasen „Scanner & Parsing“, „Transpilation“ und „Code Generation“ sowie das Fehlerbehandlungs-Verfahren für komplexe Tests:

```
graph TD
    A["Start: Katalon Test<br/>Scripts/TestName.groovy"] --> B["Phase 1: Scanner<br/>>Datei lesen"]
    B --> C["Phase 2: Parsing<br/>Kommentare entfernen<br/>Zeilen extrahieren"]
    C --> D["Regex-Filter<br/>WebUI-Methoden<br/>identifizieren"]
    D --> E{"Test zu<br/>komplex?"}
    E -->|Ja| F["Phase 4: Error<br/>Klassifizierung"]
    F --> G["Output: Fehler-Datei<br/>src/unreadable_tests/"]
    E -->|Nein| H["Phase 3: Transpilation<br/>Katalon Methoden<br/>↓<br/>Selenium WebDriver API"]
    H --> I["Mapping:<br/>TestObject → Locator<br/>GlobalVariable → Attribute"]
```

```

I --> J["Phase 4: Code Generation<br/>Python-Pytest Code<br/>Assembly"]
J --> K["Imports + Klasse +<br/>Test-Methode +<br/>Fixtures"]
K --> L["Output: Python Test<br/>src/tests/洗洗洗.py"]
L --> M["End: Generierter<br/>Pytest-Test"]

```

```

G --> N["Archiv für<br/>manuelle Überprüfung"]

```

```

style A fill:#e8f4f8
style M fill:#c8f4c8
style G fill:#f4c8c8
style H fill:#ffe8c8
style I fill:#ffe8c8

```

Das Diagramm verdeutlicht: Einfache Tests durchlaufen den vollständigen Transformationspfad. Komplexe Tests, die mit Regex-Pattern nicht zu erfassen sind, werden in ein separates Verzeichnis geschrieben und können später manuell überprüft oder angepasst werden.

5 Implementierung des Prototyps

Die Implementierung des Migrations-Werkzeugs erfolgte in Python und nutzt eine streng modularisierte Architektur, die zwischen **Build-Time** (Migrations-Pipeline) und **Runtime** (Ziel-Projektdateien) unterscheidet. Diese Trennung ermöglicht Wartbarkeit, Testbarkeit und einfache Erweiterung.

5.1 Refaktorierte Projektstruktur

Das Tool ist in folgende logische Schichten unterteilt:

```

src/
├── pipeline/                                # Build-Time: Migrations-Pipeline
│   ├── test_suite_translator.py           # Orchestrierung: Scannt Katalon Scripts
│   ├── test_transpiler.py                 # Groovy→Python Transpilation
│   ├── test_assembler.py                  # Pytest Code-Generierung
│   ├── object_repo_converter.py            # Object Repository XML→JSON Konvertierung
│   ├── global_vars_generator.py            # Global Variables Profile Generator
│   ├── variables_extractor.py              # Test Case Variables → Python
│   ├── copy_runtime_files.py               # Runtime-Dateien kopieren + Config erzeugen
│   └── __init__.py
├── runtime/                                # Runtime: Wird ins Ziel-Projekt KOPIERT
│   ├── base_test.py                       # Selenium WebDriver Base Class
│   ├── katalon_helpers.py                  # Katalon-kompatible WebDriver-Helfer
│   └── __init__.py
└── utils/                                  # Build-Time Utilities
    ├── file_utils.py                      # Datei-Operationen + Variable-Extraktion
    ├── string_utils.py                    # Identifier-Normalisierung
    ├── xml_utils.py                       # Generische XML↔JSON Konvertierung
    └── __init__.py

```

Diese Struktur trennt eindeutig:

- pipeline/: Transformation (läuft nur während Migration)
- runtime/: Abhängigkeiten (werden ins Ziel-Projekt kopiert)
- utils/: Gemeinsame Build-Time-Utilities

Das folgende Diagramm zeigt die Datenfluss-Abhängigkeiten zwischen den 7 Pipeline-Modulen:

graph TB

```

    subgraph Input["Katalon-Projekt Input"]
        SF["Scripts/*.groovy<br/>Test Cases"]
        OR["Object Repository/*.rs<br/>XML"]
        TC["Test Cases/*.tc<br/>XML"]
        PF["Profiles/default.glbl<br/>XML"]
        DF["Data Files/*.dat"]
    end

    subgraph Pipeline["Build-Time Pipeline"]
        M1["Module 1:<br/>test_suite_translator<br/>Scanner + Orchestration"]
        M2["Module 2:<br/>test_transpiler<br/>Groovy→Python"]
        M3["Module 3:<br/>test_assembler<br/>Pytest Code Gen"]
        M4["Module 4:<br/>object_repo_converter<br/>XML→JSON"]
        M5["Module 5:<br/>variables_extractor<br/>TC Variables"]
        M6["Module 6:<br/>global_vars_generator<br/>Profile Variables"]
        M7["Module 7:<br/>copy_runtime_files<br/>Assembly + Config"]
    end

    subgraph Output["Ziel-Projekt Output"]
        OUT1["src/tests/test_*.py<br/>Pytest Tests"]
        OUT2["src/object_repository/<br/>*.rs JSON"]
        OUT3["src/variables/var_*.py<br/>Test Variables"]
        OUT4["src/profiles/<br/>global_variables.py"]
        OUT5["src/runtime/<br/>base_test.py<br/>katalon_helpers.py"]
        OUT6["pytest.ini<br/>requirements.txt<br/>.gitignore"]
        OUT7["data/<br/>kopierte Testdaten"]
    end

    SF --> M1 --> M2 --> M3 --> OUT1
    M1 --> M4
    OR --> M4 --> OUT2
    TC --> M5 --> OUT3
    M1 --> M5
    PF --> M6 --> OUT4
    M1 --> M3 --> M7
    M2 --> M7
    M4 --> M7
    M5 --> M7
  
```

```
M6 --> M7
DF --> M7 --> OUT7
M7 --> OUT5
M7 --> OUT6
```

```
style Input fill:#e8f4f8
style Output fill:#c8f4c8
style Pipeline fill:#ffe8c8
```

Diese modulare Struktur gewährleistet, dass jede Pipeline-Phase isoliert getestet, validiert und bei Bedarf erweitert werden kann.

5.2 Pipeline-Phasen (Datenfluss)

Der Migrations-Prozess läuft durch 7 sequenzielle Phasen:

Phase 1 — Test Suite Scanning (test_suite_translator.py)

- Scanne source_root/Scripts nach allen .groovy Dateien
- Erhält die Katalon-Ordnerhierarchie
- Setzt bereits ein __init__.py für korrekte Python-Paketstruktur

Phase 2 — Groovy→Python Transpilation (test_transpiler.py)

- Parse jeden .groovy-Test mit Regex-Patterns
- Extrahiere WebUI-Methodenaufrufe (z.B. WebUI.click(...))
- Identifiziere Test-Objekt-Referenzen und Variablen
- Kennzeichne nicht-übersetzbare Tests (z.B. mit Handwritten-Code)

Phase 3 — Code Assembly (test_assembler.py)

- Assembliere TransPilot-Ausgaben in vollständige Python-Klassen
- Generiere Pytest-Test-Methoden mit korrekten Imports
- Erstelle Fixture-Integrationen und Assert-Statements
- Behandle Spezialfälle (String-Interpolation, Parametrisierung)

Phase 4 — Object Repository Konvertierung (object_repo_converter.py)

- Scanne source_root/Object Repository nach .rs XML-Dateien
- Konvertiere jede XML in JSON (besseres Python-Parsing)
- Gebe in destination/src/object_repository/**/*rs aus

Phase 5 — Variablen-Extraktion (variables_extractor.py)

- Scanne source_root/Test Cases nach .tc XML-Dateien
- Extrahiere Test-lokale Variablen (Typ, Name, Defaultwert)
- Generiere Python-Klassen in destination/src/variables/var_*.py

Phase 6 — Globale Variablen Generator (global_vars_generator.py)

- Lese source_root/Profiles/default.glbl (Katalon Profile XML)
- Parse GlobalVariableEntity-Einträge
- Generiere destination/src/profiles/global_variables.py

```
class GlobalVariables:
    URL = "https://..."
    USERNAME = "admin"
    PASSWORD = "secret"
```

Phase 7 — Runtime Files & Config (copy_runtime_files.py)

- Kopiere src/runtime/base_test.py ins Ziel-Projekt
- Kopiere src/runtime/katalon_helpers.py ins Ziel-Projekt
- Generiere pytest.ini, requirements.txt, .gitignore, README.md
- Kopiere Test-Daten von source_root/Data Files nach destination/data/

5.3 Kernmodule und ihre Funktionen

5.3.1 test_transpiler.py — Groovy→Python Transpildfähigkeit

Dieser Modul implementiert den Kern der syntaktischen Transformation:

```
def translate_katalon_test(katalon_test_path: str) -> Tuple[str, str]:
    """
    Read .groovy file, parse Groovy Syntax, emit Python Pytest code.
    Returns: (python_code, error_message)
    """
```

Schlüssel-Funktionen:

- parse_katalon_test(): Zerlege Groovy in Linien, finde WebUI-Methoden
- get_katalon_test_name(): Extrahiere Test-Namen aus Dateipfad
- cast_parameter(): Konvertiere numerische String-Parameter zu int/float

5.3.2 test_assembler.py — Pytest Code-Generierung

Die TestAssembler-Klasse übersetzt erkannte WebUI-Methodenaufrufe zu Selenium-Äquivalenten:

```
class TestAssembler:
    translate_methods_list = [
        "openBrowser", "click", "setText",
        "verifyElementPresent", "delay", ...
    ]

    def click(self, to, *args):
        self.file_content_tests.append(f"{to}.click()")

    def setText(self, to, text, *args):
        self.file_content_tests.append(
            f"{to}.clear()\n{to}.send_keys({text})"
        )
```

Jede Methode kennt die Katalon-Signatur und erzeugt das Python-Äquivalent.

5.3.3 object_repo_converter.py — Object Repository Migration

Konvertiert Katalon's Object Repository Format:

```
def create_object_repository(source_root: str, destination_root: str):
    # Scanne source_root/Object Repository
    # Für jede .rs XML-Datei:
    #   → write_xml_to_json(xml_path, json_path)
```

Der Output ist eine JSON-Struktur, die von katalon_helpers.py im Ziel-Projekt gelesen wird:

```
{
  "WebElementEntity": {
    "selectorMethod": "XPATH",
    "selectorCollection": {
      "entry": [
        { "key": "XPATH", "value": "//input[@id='email']" }
      ]
    }
  }
}
```

Das folgende Diagramm zeigt den Transformationsprozess von Katalon Object Repository zu JSON:

```
graph LR
    A["Input:<br/>Object Repository/<br/>*.rs XML"] --> B["Scanner:<br/>Alle .rs  
Dateien<br/>verzeichnis-rekursiv"]
    B --> C["XML Parser:<br/>xmldict"]
    C --> D["Extract:<br/>WebElementEntity<br/>selectorMethod<br/>  
>selectorCollection"]
    D --> E["Transform:<br/>Nested XML →<br/>Flache JSON<br/>Struktur"]
    E --> F["Map Locators:<br/>XPATH/CSS/ID<br/>zu Selenium<br/>By-Strategien"]
    F --> G["Output:<br/>src/object_repository/<br/>*.rs JSON"]

    style A fill:#e8f4f8
    style G fill:#c8f4c8
    style C fill:#ffe8c8
    style E fill:#ffe8c8
    style F fill:#ffe8c8
```

Die konvertierten JSON-Dateien werden zur Laufzeit von katalon_helpers.py gelesen, um Locator-Strategien zu bestimmen.

5.3.4 global_vars_generator.py — Globale Variablen

Liest .glbl XML-Profil und generiert Python:

```
def create_global_variables_file(source_root: str, dest_root: str):
    glbl_path = f"{source_root}/Profiles/default.glbl"
    with open(glbl_path) as f:
        profile_data = xmldict.parse(f.read())

    # Extract GlobalVariableEntity items
    for entity in profile_data["GlobalVariableEntities"]["GlobalVariableEntity"]:
```

```

name = normalize_identifizier(entity["name"])
value = to_python_literal(entity["initValue"])
# Write: name = value

```

Das folgende Diagramm zeigt die Generierung von globalen Variablen und die Extraktion von Test-lokalen Variablen:

graph LR

```

A1["Input:<br/>Profiles/<br/>default.glbl"]
A2["Input:<br/>Test Cases/<br/>*.tc XML"]

A1 --> B1["Parser:<br/>xmltodict<br/>GlobalVariableEntity"]
A2 --> B2["Parser:<br/>xmltodict<br/>variable tags"]

B1 --> C1["Extract:<br/>Name +<br/>Initial Value +<br/>Type"]
B2 --> C2["Extract:<br/>Variable Name +<br/>Default Value +<br/>Type Info"]

C1 --> D1["Normalize:<br/>SCREAMING_SNAKE_CASE<br/>Python Literal"]
C2 --> D2["Normalize:<br/>snake_case<br/>Class Attributes"]

D1 --> E1["Output:<br/>src/profiles/<br/>global_variables.py"]
D2 --> E2["Output:<br/>src/variables/<br/>var_*.py"]

style A1 fill:#e8f4f8
style A2 fill:#e8f4f8
style E1 fill:#c8f4c8
style E2 fill:#c8f4c8
style D1 fill:#ffe8c8
style D2 fill:#ffe8c8

```

Die generierten Variablen-Module stehen allen Tests zur Verfügung und können über Import oder direkt über ihren Klassen-Namespace angesprochen werden.

5.3.5 runtime/base_test.py — Selenium Base Class

Die Basis-Testklasse für alle generierten Tests:

```

class BaseTest:
    @classmethod
    def setup_class(cls):
        cls.driver = cls.chrome_configurations()
        cls.driver.maximize_window()
        cls.driver.get(GlobalVariables.URL)

    @classmethod
    def teardown_class(cls):
        if hasattr(cls, "driver"):
            cls.driver.quit()

```

5.3.6 runtime/katalon_helpers.py — Katalon-kompatible Helfer

Stellt Funktionen bereit, die von generierten Tests aufgerufen werden:

```
def find_katalon_test_object(driver, path: str) -> WebElement:
    """Locate element via converted Object Repository JSON."""
    # Read JSON from src/object_repository/
    # Extract selector (CSS, XPATH, etc.)
    # Return driver.find_element(...)

def find_katalon_test_data(csv_name, column, row) -> str:
    """Read test data from CSV files in data/ folder."""
```

Diese Module werden ins Ziel-Projekt kopiert und sind Runtime-Abhängigkeiten.

5.4 Fehlerbehandlung und Robustheit

Das Tool implementiert mehrere Validierungsebenen:

1. **Transpilation-Fehler:** Tests mit Handwritten-Code (z.B. if-Statements, String-Variablen-Deklarationen) werden als „not automatically translateable“ gekennzeichnet und in `src/unreadable_tests/` geschrieben
2. **Referenz-Validierung:** `check_params_for_specialties()` validiert dass:
 - Alle `findTestObject('...')` Referenzen im Object Repository existieren
 - Alle `findTestData('...')` Referenzen in Data Files existieren
3. **Syntax-Validierung:** Generierter Python-Code wird durch AST-Parsing validiert
4. **Import-Validierung:** Vor der Generierung wird geprüft, dass alle notwendigen Module verfügbar sind

5.5 Konfigurierbarkeit und Erweiterung

Das System ist für Erweiterungen ausgelegt:

- Neue WebUI-Methoden: Einfach neue Methoden zu `TestAssembler` hinzufügen
- Neue Datenquellen: Neue Extraktoren für Custom Keywords oder Plugins
- Verschiedene Browser: `base_test.py` kann für Firefox, Edge angepasst werden
- Pytest-Hooks: `conftest.py` in Ziel-Projekt kann Hook-Integration vornehmen

```
def write_test_file(test_content, test_name, output_dir):
    filepath = os.path.join(output_dir, f"test_{test_name}.py")
    with open(filepath, 'w', encoding='utf-8') as f:
        f.write(test_content)
```

Fügt auch automatisch:

- Import-Statements (`pytest`, `selenium`, etc.)
- Fixture-Definitionen in `conftest.py`
- Test-Daten-Referenzen ein
- Konfigurationsdetails in `pytest.ini` oder `pyproject.toml`

5.5.1 base_test.py — Pytest-Basis-Klasse

Definiert eine Basis-Testklasse, die von allen migrierten Tests erbt:

```
class BaseTestCase:
    @pytest.fixture(autouse=True)
    def setup(self):
        self.driver = self.get_webdriver()
        self.wait = WebDriverWait(self.driver, timeout=10)

    def teardown(self):
        self.driver.quit()
```

6 Evaluation & Results

Test Suite Key	Katalon Execution (s)	Selenium Execution (s)
TS-01: Core Authentication	14.2	11.5
TS-02: Inventory Checkout	32.8	24.1
TS-03: Reporting Dashboard	45.1	41.9

Tabelle 1: Runtime performance and execution velocity comparisons between the legacy system and the migrated prototype. (Source: Compiled from test logs)

The validation metrics recorded inside Tabelle 1 show the functional performance of direct WebDriver implementation.

7 Discussion & Future Work

[Evaluate technical constraints, limitations, and future scalability to frameworks like Playwright or Cypress.]

8 Conclusion / Zusammenfassung

[Summarize final project milestones.]

9 Appendix / Anhang

[Attach sample XML files and extracted parsed code structures here.]

10 Declaration of Originality / Eigenständigkeitserklärung

I hereby declare that:

- I have written this academic thesis independently and without any unauthorized assistance,
- I have not used any sources or aids other than those explicitly specified,
- I have clearly marked all passages taken verbatim or in spirit from the referenced sources as citations,
- This work has not been submitted in the same or a similar form to any other examination authority.

.....

.....

Berlin, Date

Konstantin Regenhardt

Quellenverzeichnis

- [1] Katalon, Inc., „Katalon | The AI Platform for Software Quality“. Zugegriffen: 21. Juli 2026. [Online]. Verfügbar unter: <https://katalon.com/>
- [2] International Software Testing Qualifications Board, „ISTQB Glossary“. Zugegriffen: 22. Juli 2026. [Online]. Verfügbar unter: <https://glossary.istqb.org/>
- [3] C. Shapiro und H. R. Varian, *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business School Press, 1998.
- [4] A. Sahay, A. Indamutsa, D. Di Ruscio, und A. Pierantonio, „Supporting the Understanding and Comparison of Low-Code Development Platforms“, in *Proceedings of the 46th Euro-micro Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2020, S. 171–178. doi: 10.1109/SEAA51224.2020.00036.
- [5] pytest development team, „Pytest Documentation: Configuration“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/customize.html>
- [6] pytest development team, „Pytest Documentation: Good Integration Practices“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/explanation/goodpractices.html>
- [7] Python Packaging Authority, „Python Packaging User Guide: src layout vs flat layout“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/>
- [8] pytest development team, „Pytest Documentation: Fixtures Reference“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/fixtures.html>

Erklärung zur selbstständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne fremde Hilfe selbständig verfasst und nur die angegebenen Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Ort

Datum

Unterschrift im Original